Manual de Usuario - Analizador Markdown

1. Descripción

El proyecto analiza un documento Markdown en tres fases:

1. Análisis léxico.
2. Análisis sintáctico.
3. Análisis semántico.

Después genera reportes HTML a partir del árbol construido.

2. Requisitos

- Python 3.10 o superior.
- Tkinter, incluido normalmente en Python para Windows.
- Un archivo Markdown con extensión `.md`.

No se requieren librerías externas.

3. Ejecución gráfica

Desde la carpeta raíz del workspace, ejecute:

```
python .\Proyecto2\app.py
```

Se abrirá la ventana del analizador.

Analizar un archivo

1. Presione **Seleccionar**.
2. Seleccione un archivo `.md` dentro de `Proyecto2/test` o cualquier otra carpeta.
3. Presione **Analizar**.
4. Consulte el estado, las estadísticas y los errores mostrados.

También puede iniciar la GUI con un archivo seleccionado:

```
python .\Proyecto2\app.py .\Proyecto2\test\Prueba.md
```

El archivo quedará seleccionado y deberá presionar **Analizar**.

Generar reportes

Después de analizar el archivo, presione **Generar HTML** y seleccione una carpeta de salida.

Se crearán tres archivos:

- `documento_renderizado.html`: traducción del documento a HTML.
- `estadisticas.html`: cantidades de palabras, tokens, bloques y elementos Markdown.
- `tabla_contenido.html`: índice con enlaces hacia los encabezados.

4. Ejecución en consola

Para analizar sin abrir la GUI:

```
python .\Proyecto2\app.py .\Proyecto2\test\Prueba.md --consola
```

Para analizar y generar un archivo HTML:

```
python .\Proyecto2\app.py .\Proyecto2\test\Prueba.md --consola --html
.\Proyecto2\test\resultado.html
```

5. Elementos reconocidos

El analizador reconoce:

- Encabezados de 1 a 6 niveles: `# Título`.
- Viñetas: `-`, `*` y `+`.
- Listas anidadas mediante indentación.
- Citas: `>` y citas anidadas.
- Bloques de código con tres backticks.
- Reglas horizontales con tres o más guiones.
- Negrita: `**texto**`.
- Cursiva: `*texto*`.
- Código inline: `` `código` ``.
- Texto normal.
- Líneas vacías como separadores de párrafos.

6. Errores

Los errores se muestran en la ventana o en la consola. El sistema distingue entre:

- Errores léxicos: problemas al reconocer caracteres o tokens.
- Errores sintácticos: estructura Markdown incompleta o incorrecta.
- Errores semánticos: estructura reconocida que no cumple las reglas del proyecto.

Aunque existan errores, el sistema conserva la información disponible y puede generar un HTML de diagnóstico.

7. Archivos principales

- `Proyecto2/app.py`: punto de entrada.
- `Proyecto2/vista.py`: interfaz Tkinter.

- `Proyecto2/controlador.py`: coordinación y generación de reportes.
- `Proyecto2/lexer.py`: análisis léxico.
- `Proyecto2/parser.py`: análisis sintáctico.
- `Proyecto2/semantic.py`: análisis semántico.